ePMapping: A Staged Rewriting Framework for Post-Mapping Area Optimization

***Abstract*—Technology-independent logic optimization does not necessarily minimize standard-cell area. We present ePMapping, a post-mapping optimizer under the input delay bound. Joint AO21 chain transformations share factors across roots. Local rewriting combines cell matching, database lookup, Shannon/Davio synthesis and bounded equality saturation. Progressive extraction builds reusable arrival-independent profiles; live pricing accounts for physical phase reuse. Rebuilt-netlist checks enforce area and timing constraints. We evaluate common-input optimization against ABC, recovery after AIG preprocessing, and operator ablations. Across sixteen preprocessed designs, selected outputs change mean area and delay by -7.11% and -7.43%. Live-query and truth-table reuse reduce repeated work; a support-ranked pivot budget bounds decomposition search. Cold and warm measurements are reported separately, and deadline termination remains marked incomplete.**

***Index Terms*—Logic synthesis, post-mapping optimization, standard-cell mapping, e-graph, don't cares.**

I. Introduction

Logic synthesis commonly optimizes an and-inverter graph (AIG) [1] or majority-inverter graph (MIG) [2], [3] before technology mapping [4]. Graph size is only a proxy for cell area: equivalent graphs can map differently because library functions, pin delays and inversion costs differ.

Post-mapping resubstitution uses dependency cuts and mapped-subnetwork databases [5], [6]; equality saturation explores equivalent implementations [7]–[10]. We combine local cover diversity with sharing across roots.

Compact factorizations can miss library-friendly Shannon/Davio forms. More extraction capacity cannot introduce absent decompositions. Across roots, isolated costs can obscure shared factors.

ePMapping addresses both scopes (Fig. 1). Each round jointly evaluates AO21 blocks, then combines cell matching, database lookup and structural synthesis. Bounded equality saturation follows when these yield no admissible proposal. Live required times permit slack use under the initial delay bound; rebuilt netlists guard acceptance.

The contributions are:

- staged cut search with Shannon/Davio alternatives, reusable arrival-independent e-graph pools and live phase pricing;
- joint AO21 block composition and cross-root reuse, accepted by total mapped area;
- a three-part evaluation of common-input optimization, recovery after preprocessing, and component area/runtime trade-offs.

II. Preliminaries and Objective

A. Mapped Netlists and Cuts

A technology library $\mathcal{L}$ supplies a Boolean function, area and physical input-pin delays for each standard cell. Technology mapping covers a technology-independent subject graph with these cells, producing a mapped netlist N. Let $V_{\mathrm{cell}}(N)$ denote its live physical cells and $O(N)$ its primary outputs. We use

$$A(N) = \sum_{v \in V_{\mathrm{cell}}(N)} a(v), \qquad D(N) = \max_{o \in O(N)} \tau(o), \tag{1}$$

where $a(v)$ is cell area and τ is arrival time. Pin delays follow the load-independent gain-based model [4]. Inversions are materialized as inverter cells and included in area and timing. Native and serialized netlists may differ in interface buffers and duplicate-output aliases; the experimental comparison accounts for these differences.

Given the input mapped netlist N_0, define $A_0 = A(N_0)$ and $D_0 = D(N_0)$. Our optimization problem is

$$\min_{N} A(N) \quad \text{subject to} \quad D(N) \le D_0, \quad N \equiv N_0. \tag{2}$$

Delay is the secondary objective on an area tie. Internal arrival times may increase when sufficient slack remains under D_0.

For a gate x, the *transitive fanin* (TFI) contains the nodes feeding x. Its *maximum fanout-free cone* (MFFC) is the logic used exclusively by x, including the root. A *structural cut* $\mathcal{C} = (x, L)$ has leaves L intersecting every primary-input-to-x path; its cut function $f_{\mathcal{C}}$ expresses x over those leaves. A dependency cut can instead use non-structural leaves over which f_x is expressible [6].

The satisfiability don't-care (DC) set contains leaf minterms that cannot occur. A replacement agreeing with $f_{\mathcal{C}}$ on all remaining *care* minterms preserves the root function. Conservative unreachable-minterm estimates therefore allow functional flexibility without requiring observability don't cares.

B. Covers and E-Graphs

A cut *cover* H is a cell directed acyclic graph (DAG) implementing an admissible function over L. Its representation records shared nodes, input and output polarities, and physical pins. Generators provide an area estimate $\widehat{A}(H)$; caller-side evaluation computes the root arrival from live leaf arrivals. The net area change additionally depends on which existing cells become unused and which cover nodes can be reused. Section III-F describes both local ranking and rebuilt-netlist validation.

An *e-graph* [11] groups equivalent expressions into e-classes. Equality saturation [12] expands these classes through rewriting, after which extraction selects an implementation under a

Algorithm 1: A round of ePMapping.

Input: Mapped netlist N, initial delay bound D_0, library $\mathcal{L}$, maximum cut leaves K, retained-cut limit B, e-graph threshold θ.

save a round backup of N and its (A, D) pair;
compose and share eligible AO21 chain blocks;
commit the transaction only if rebuilt area decreases and $D \le D_0$;
initialize a lazy cut manager for N with bounds K, B;
foreach *live gate x in reverse topological order* **do**
    $best \leftarrow \emptyset$;
    foreach *nontrivial structural cut $\mathcal{C} = (x, L)$, $|L| \le K$* **do**
        compute $f_{\mathcal{C}}$ and its care mask;
        $S \leftarrow \textsc{SingleCell}(\mathcal{C}) \cup \textsc{Database}(\mathcal{C}) \cup \textsc{EligibleSynthesis}(\mathcal{C})$;
        $V \leftarrow$ proposals passing live phase, cost and timing checks;
        if *$V = \emptyset$ and $\mathcal{C}$ is the MFFC boundary cut and $A_{\mathrm{rem}}(x, L) \ge \theta$* **then**
            load or construct profiles; add feasible covers to V;
        update *best* with the best proposal in V;
    if *$best \neq \emptyset$ and it passes functional and cycle checks* **then**
        substitute x; update fanouts and timing; invalidate affected cuts;
rebuild N; restore the round backup if $D > D_0$ or (A, D) worsens;

cost model. ePMapping uses *egg* [13] for bounded saturation and heuristic extraction. Extracted covers undergo the same physical feasibility checks as covers from the other generators.

III. Staged Post-Mapping Rewriting

A. Overview

The two stages address different optimization scopes. Chain composition is evaluated jointly across roots; cut rewriting selects a cover for one root at a time. After the chain stage, live gates are visited in reverse topological order using a lazy structural-cut manager. Each substitution updates fanouts, levels and arrivals and invalidates affected cuts.

B. Chain Blocks and Shared Factors

The chain stage exposes reuse that cannot be priced adequately by independent root substitutions. Consider a positive AO21 chain. An AO21 cell implements $y = (a \wedge b) \vee c$. Each chain step is $F_i(z) = (p_i \wedge z) \vee q_i$. A segment represents (P, Q) with $F(z) = (P \wedge z) \vee Q$, and exact composition is

$$(P, Q) \circ (p, q) = (P \wedge p,\ Q \vee (P \wedge q)). \tag{3}$$

Equation (3) follows from Boolean distributivity and remains valid when coefficient signals are dependent. The optimization opportunity arises when identical products or segments can be reused by different roots.

Chains follow an AND input through single-fanout internal AO21 cells and stop at outputs or branching boundaries. Complemented fanin edges are excluded, although coefficient signals may be driven by explicit inverter cells. Traversal collects 6–64 steps per eligible segment, ordered from the root toward the boundary. Recursion splits off two-step blocks with balanced subdivision inside a block, terminating at $(0, z)$. This construction balances each block rather than the entire chain. Products use AND2–AND5 cells with arrival-aware physical pin assignment.

Product memo keys contain sorted unique factor signals; segment keys contain ordered coefficient pairs. Memoization spans roots, so identical factors and segments can be reused. All root proposals are constructed before substitution and treated as one transaction. Rebuilding counts every surviving cell, including the shared factors and residual functions. The transaction is retained only if total area strictly decreases and $D \le D_0$; otherwise the original netlist is restored. Subsequent cut rewriting may further simplify the resulting structures.

C. Cover Representation and Feasibility

All generators return a cell DAG with explicit polarities and physical pin assignments. Before phase resolution, a node v with fanins u_i, pin delays $d_{v,i}$, input inversions $b_{v,i}$ and output inversion o_v has arrival

$$\tau_H(v) = \max_i \{\tau_H(u_i) + d_{v,i} + b_{v,i} d_{\mathrm{inv}}\} + o_v d_{\mathrm{inv}}. \tag{4}$$

Here $\tau_H(\ell) = \tau(\ell)$ is the current arrival of leaf ℓ. The binary flags $b_{v,i}$ and o_v indicate input and output inversion, respectively, and d_{inv} is inverter delay. Pin delays are resolved by physical pin name, including permuted variants. Polarity flags describe inversions before emission; explicit inverter nodes are not charged a second time. Shared cover nodes are emitted once. Final area is the live-cell sum in Equation (1). Area and timing are evaluated in the library model of Equation (1).

Symbolic phase resolution precedes local selection and follows the same rules as materialization. If $u = \neg v$, a request for $\neg u$ uses v. An existing INV fanout can also supply $\neg u$, including a shared gate inside the root's fanin. It must have a live path to an output and must not depend on the replacement root. Explicit INV nodes and polarity flags use the same resolver; no cut is widened.

Let M be the cut's removable cells, J_H the existing cells retained by the resolved cover, and V_H^{new} its reachable new cells. The incremental implementation estimate is

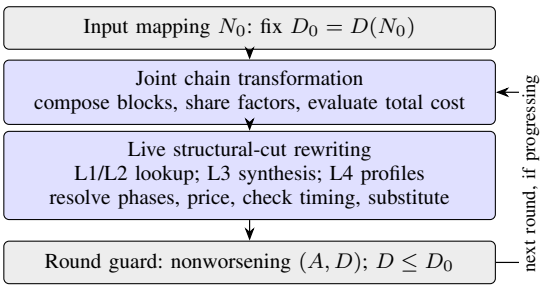
$$\widehat{A}_{\mathrm{ctx}}(H) = \sum_{v \in V_H^{\mathrm{new}}} a(v) + \sum_{v \in J_H \cap M} a(v). \tag{5}$$

Existing shared cells outside M add no area; retaining a removable gate is charged. Cancelled and unreachable new nodes are excluded. Timing is propagated on the resolved DAG using actual arrivals at reused signals. Rebuilt netlist measurements guard the combined result. The winning cover is checked on care minterms; final CEC validates rewrite interactions.

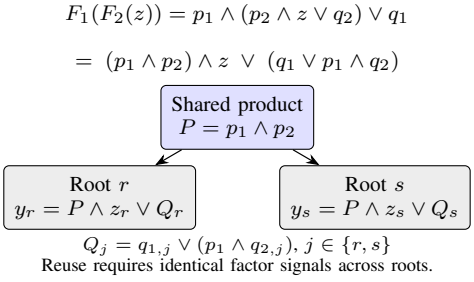
Care computation projects a bounded reconvergence simulation onto cut leaves. Free inputs over-approximate surrounding logic, so only conservatively unreachable minterms become DCs. Cuts outside the window remain fully specified. SAT refinement is disabled. L1, L2 and SOP factoring can exploit partial functions; cofactor candidates and L4 use the fully specified function.

D. Four Complementary Cover Generators

a) L1: indexed single-cell matching: The library index includes input permutations and polarities and probes both output phases. Partial-function matching enumerates completions under free-minterm and entry caps.



(a) Two scopes of rewriting.



(b) Two-step composition exposes a reusable product.

Fig. 1. Overview of ePMapping. (a) Joint chain transformation precedes local cut rewriting; both use the fixed initial delay bound and a final round check. (b) Two-step composition exposes a product P reusable across roots with different residual functions Q_j. Acceptance accounts for all cells implementing the product, residual functions and outputs.

TABLE I. Cover generators and their practical limits. L1–L3 compete on eligible cuts; L4 is a fallback after acceptance filtering.

Generator	Mechanism	Limit
L1 single cell	indexed function, input phases and permutations	one cell; bounded DC completions
L2 database	pre-mapped four-input cell DAGs	stored functions and entry limits
L3 synthesis	SOP/DSD, Shannon and Davio decompositions	bounded support and heuristic mapping
L4 e-graph	AIG rewrites and library cover rules	rewrite reachability, saturation budget and heuristic extraction

b) L2: mapped sub-network lookup: ABC's `super` builds the database offline. A separate library supplies lookup and internal cut mapping without changing initial `emap`; noDB removes both uses.

c) L3: structural synthesis: Sum-of-products (SOP) factoring and disjoint-support decomposition (DSD) build AIGs and map them to the library. A compact factorization is only one choice. For $f = f_{\mathcal{C}}$ and a retained pivot $\ell \in L$, let $f_b = f|_{\ell=b}$. We independently construct and map

$$\begin{aligned} f &= (\ell \wedge f_1) \vee (\neg\ell \wedge f_0), \\ f &= f_0 \oplus [\ell \wedge (f_0 \oplus f_1)] \\ &= f_1 \oplus [\neg\ell \wedge (f_0 \oplus f_1)]. \end{aligned} \tag{6}$$

The first is Shannon decomposition; the latter two are positive/negative Davio forms. Cofactors and Boolean differences are SOP-factored before mapping. These candidates implement the fully specified cut function and are checked over its entire truth table. They compete with the other sources under the same timing bound.

We retain two pivots per decomposition family. Let $s(g)$ be the number of variables in the support of g, and let $\Delta_\ell = f_0 \oplus f_1$. Shannon ranks pivots by $s(f_0) + s(f_1)$; positive and negative Davio use $s(f_0) + s(\Delta_\ell)$ and $s(f_1) + s(\Delta_\ell)$, respectively. Ties follow leaf order; retained pivots are processed in their original order. This structural budget reduces candidate construction but can omit useful covers. Timing feasibility is checked after mapping, not inferred from support size.

For example, $d \wedge (\neg c \vee a) \wedge (c \vee b)$ equals $(d \wedge c \wedge a) \vee (d \wedge \neg c \wedge b)$. The latter can use an AO33 with d connected to two pins. The six-pin cell then implements a four-variable function, a binding not enumerated by the direct single-cell matcher. Decomposition changes the available library covers rather than just enlarging an extraction front.

E. Equality Saturation for Cover Generation

a) Language and rules: Each input cell is expanded through its genlib expression into an AIG. Gate symbols and AIG nodes coexist; truth-table analysis checks merged e-classes. Static rules cover double-negation elimination/insertion, AND commutativity and reassociation, complements, idempotence and constants. Library rules replace matching AIG expressions with cells. Cell-to-AIG expansion is omitted because the seed is already expanded. The budget is two iterations and 500 e-nodes; this local search does not systematically enumerate the decompositions in Equation (6).

b) Extraction: Extraction is independent of current leaf arrivals. A worklist retains cell DAGs with alternative pins and child realizations. A narrow pass first keeps one area-oriented realization per e-class, using at most one quarter of the combination allowance and half the visit allowance. The wider pass starts from these realizations and receives all unused work. Both passes together obey the original limits; the narrow pass reserves inexpensive coverage before wider Cartesian enumeration. Structural interning shares identical nodes. Child dependencies containing the current e-class are excluded to bound recursive unrolling. Raw AIG nodes require AND and inverter cells.

For a cover H, let $U_H \subseteq L$ be its used inputs and $p_i(H)$ the longest input-to-root delay, including polarity costs. Its current arrival is

$$T(H; \boldsymbol{\tau}) = \max_{i \in U_H} \{\tau(\ell_i) + p_i(H)\}. \tag{7}$$

Constant covers have zero arrival in this model. The stored profile is $(\widehat{A}(H), p_0(H), \ldots, p_{|L|-1}(H))$. Unused coordinates are ignored in timing and treated as $-\infty$ for dominance. Root pruning compares area and every delay coordinate, avoiding scalar zero-arrival pruning. Distinct intermediate structures can survive standalone dominance because parents may share their nodes. Caps make this an incomplete candidate pool; intrinsic profiles also omit external reuse.

Persistent keys include the expanded AIG seed, variable order, library, extraction version, limits and retention settings,

excluding arrivals and required times. Hits load DAGs and vectors; misses saturate and extract. Pin variants are enumerated only for cells in the bounded DAG; invalid records are rebuilt. Empty pools retry single-realization extraction. With contextual pricing, intrinsic rounded-arrival filtering is bypassed: phase resolution may improve timing. Selection uses unrounded live costs; bounded intrinsic pruning can still omit useful context-dependent covers.

F. Acceptance and Iteration

Backward required times implement Equation (2). For an output, $R(o) = D_0$; for an internal signal,

$$R(u) = \min_{(u,v,i)} \{R(v) - d_{v,i} - b_{v,i} d_{\text{inv}}\}, \tag{8}$$

with output inversions deducted at the boundary. The minimum ranges over live fanout pins. A cover is timing-feasible when its recomputed root arrival does not exceed $R(x)$ within tolerance. Required-time caches are invalidated after substitution, so a noncritical rewrite may consume current slack without relaxing D_0.

Across cuts, candidates rank by $\widehat{A}_{\text{ctx}}(H) - A_{\text{rem}}(x, L)$, then root arrival and structural level. Here $A_{\text{rem}} = \sum_{v \in M} a(v)$; shared existing logic is not assumed removable. Functional and cycle checks precede substitution. A rebuilt round must respect D_0 and not worsen the quantized (A, D) pair, or it is restored.

The bound spans all rounds. A faster but larger result is rejected; delay and then level break area ties. Area and delay keys are quantized to 10^{-4} and 10^{-3}, respectively. Roots without admissible covers are skipped. The evaluated method uses neither whole-window synthesis nor unmap/remap exploration. Reconvergence-window simulation is used only to compute cut care sets.

Rounds stop on no substitution, no objective progress, or the round cap.

G. Reusing Construction Work

Native synthesis caches support, ordered truth tables, mode and objective; SOP keys include DCs and cofactor keys the pivot. A second cache reuses mapped DAGs and full-function proofs across cofactor requests with identical pre-mapping AIG states. Keys preserve PI/PO order, every node slot, edge phases, reference counts and traversal state. Library and mapper settings are fixed. No contextual price is cached across network changes. Request caches retain entries across rounds; transient mapped networks share an owned immutable library.

For an area-first cut, sum root-reachable non-inverter cell areas, counting shared nodes once. This lower bound excludes inverter costs and retained removable gates; other cells are emitted anew. Live pricing is omitted only when the bound exceeds a feasible incumbent area by more than 10^{-3}, or the original cone area if no incumbent exists. Area ties remain eligible. Fresh SOP/DSD and cofactor covers are priced once per unchanged cut.

Inverse-phase queries cache both a reusable live inverter and a negative result. Entries are valid only for one root, network revision and node-slot count; substitution, rebuilding or candidate-node insertion invalidates them. An inverter at a strictly lower structural level cannot depend on the root; remaining cases retain the dependency traversal. Cut simulation decodes gate functions and fanins once, reuses scratch storage, and supplies the cached truth table to repeated requests. These mechanisms reuse computation; the pivot budget separately restricts search.

Every statistical deferral is rechecked in its original position before root commitment. Only strict area dominance skips live pricing among the generated covers.

Periodically after substitutions and at round ends, a rebuilt copy is saved if it meets the original area/delay bounds and improves the best quantized objective. The initial mapping is saved first. A deadline leaves the search incomplete; independent CEC and serialized-netlist audits admit the saved output. Snapshot construction and writing consume the post-mapping budget.

TABLE II. One round on byte-identical initial netlists and the same six-input sublibrary. Costs are normalized to the native initial costs of each tool; ABC uses raw outputs. †: delay exceeds its initial bound; TO: 1000 s. Times in seconds exclude CEC; ours gives cold/warm. Ave. Ratio reports mean ratios and times over the 13 ABC-completed designs.

Circuit	ABC mfs3			Ours		
	A/A_0	D/D_0	t	A/A_0	D/D_0	t_c/t_w
aes_core	0.9731	1.1333†	32.5	0.9825	0.9999	186.0/59.7
arbiter	1.0000	1.0000	96.3	0.6282	0.5655	6.6/6.5
div	TO	TO	1000	0.9606	0.9954	407.1/342.4
ethernet	0.9935	1.1019†	52.6	0.9863	0.9812	150.8/69.8
iwls05_i2c	0.9601	0.9843	0.9	0.9259	0.9901	14.8/2.4
mem_ctrl	TO	TO	1000	0.9591	0.9896	376.2/174.6
RISC	0.9715	1.6875†	91.9	0.9446	0.9977	341.8/128.2
sasc	0.9816	1.0000	0.1	0.9886	1.0000	1.6/0.8
simple_spi	0.9730	1.0000	0.1	0.9766	0.9022	6.6/1.4
spi	0.9786	1.0510†	7.6	0.9550	0.9912	24.4/8.1
sqrt	TO	TO	1000	0.8857	0.9996	90.1/80.9
systemcaes	0.9704	1.0904†	17.6	0.9753	0.8132	35.0/17.3
systemcdes	0.9251	1.2116†	1.8	0.9382	0.9947	23.3/13.0
tv80	0.9301	1.1757†	38.4	0.9491	0.9805	85.9/26.6
usb_funct	0.9745	0.9968	3.5	0.9482	0.9392	93.8/26.5
usb_phy	0.9035	1.0000	0.1	0.8832	0.9852	4.3/1.1
Ave. Ratio	0.9642	1.1102	26.4	0.9294	0.9339	75.0/27.8

IV. Experimental Results

A. Setup

We evaluate sixteen designs with the ASAP7 genlib [14], reporting area in μm^2 and worst-output delay in ps. We use C++/mockturtle [15], Rust/*egg* [13], and ABC [16].

All experiments fix the initial delay bound across rounds (Equation (2)), with whole-window synthesis and unmap/remap exploration disabled.

For `mfs3`, both tools use the same 40-entry subset with at most six inputs per cell, and byte-identical area-oriented `emap` inputs. The evaluated `mfs3` implementation restricts cell truth tables and node simulation to six inputs. Preprocessing experiments and ablations use all 47 entries for initial mapping and optimization.

The direct input N_i maps the raw AIG without ABC optimization. For N_e, the flow repeats `fraig; resyn2rs;`

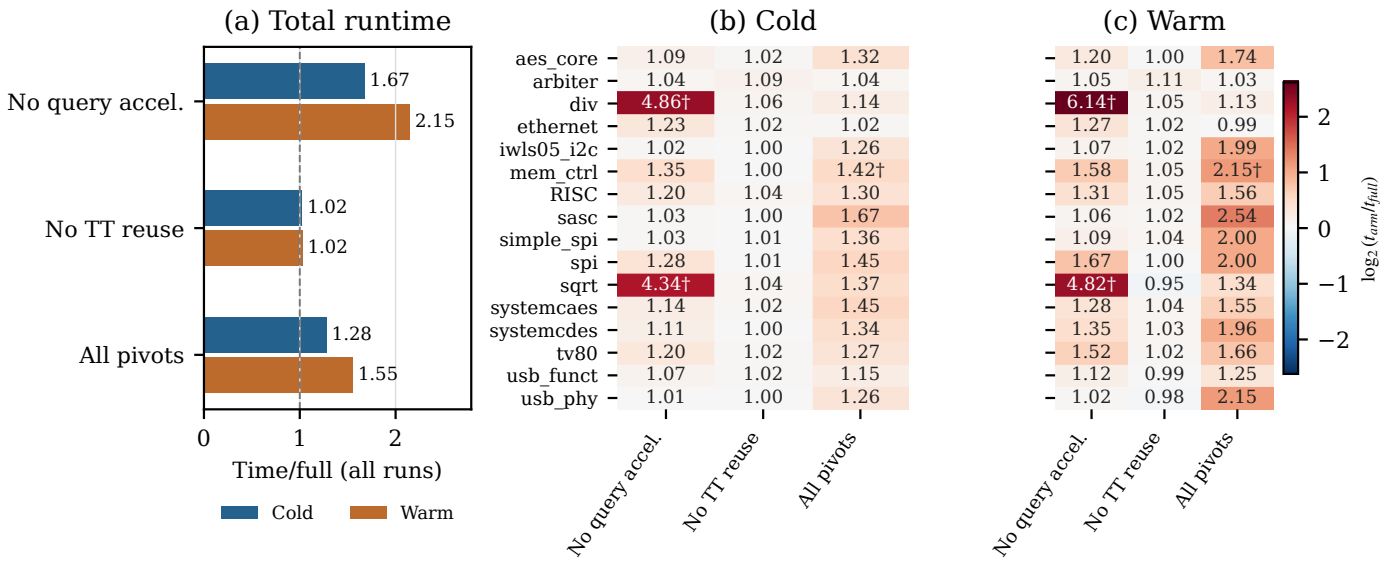


Fig. 2. Query, truth-table and pivot-control runtime relative to full, cold/warm: aggregate optimizer-time ratios and per-design ratios. Colors encode the logarithm of each ratio. † marks at least one TO; equal elapsed budgets do not imply equal completed work. Selected-output area effects and timeout counts appear in TABLE IV.

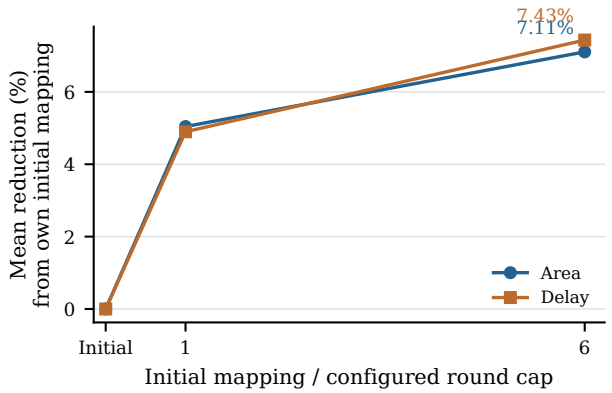


Fig. 3. Selected-output reductions from N_e at caps 1 and 6. Cold/warm curves separate when different; intermediate caps were not measured.

`fraig; compress2rs` until the AND count stops decreasing, with a 20-iteration cap, then applies area-oriented `emap`. Let $A_e = A(N_e)$ and $D_e = D(N_e)$.

Cuts use $K = 6$, $B = 12$ and $\theta = 0$. Care uses an eight-input reconvergence window and 60 s cumulative budget, then full care; SAT refinement is off. The three-level database has area cap 0.50. SOP/DSD starts at four leaves; Shannon/Davio at three, with two ranked pivots per family. Each native structure cache holds 262144 entries. Independent polarity factoring and exact AIG synthesis are off. L4 allows two iterations, 500 e-nodes, eight realizations per class, 20000 combinations and 4000 visits. Chain blocks have two steps and minimum length six. Round caps allow early stopping.

Experiments run on two Xeon Gold 6226R sockets (32 cores), Ubuntu 22.04.5, GCC 11.2.0, Rust 1.99.0 and ABC 1.01. A shared host runs up to twelve single-worker ePMapping optimizations and six independent ABC jobs. Both tools have a 1000 s post-mapping limit. For ePMapping this includes optimizer preparation, all rounds and final rebuild, excluding preprocessing, mapping and CEC. An outer 3600 s guard covers setup and validation.

TABLE III. Preprocessed mappings, at most six rounds. ASPDAC'25 costs are reconstructed from reported starts and rounded changes; starts differ. Area: μm^2; delay: ps. t_c/t_w: cold/warm optimizer seconds, excluding preprocessing, mapping and CEC. All runs finish within 1000 s. Differing costs are stacked cold above warm. Ave. Ratio averages ratios to the reconstructed reference over all 16 designs.

Circuit	ASPDAC'25		Initial		Ours		Time (s)
	Area	Delay	A_e	D_e	Area	Delay	t_c/t_w
aes_core	1086.57	425.61	1029.98	410.72	996.04	407.66	420.6/156.4
arbiter	268.82	604.27	557.72	999.87	223.64	261.39	19.5/19.7
div	1176.29	60254.25	1184.50	53105.06	1182.38	53024.38	210.2/170.1
ethernet	3008.41	588.34	2541.41	543.88	2531.03	543.56	466.8/330.6
iwls05_i2c	48.96	288.07	47.66	303.61	45.11	290.02	18.8/4.9
mem_ctrl	1799.15	1435.03	1895.40	1519.75	1818.16	1461.58	713.5/463.9
RISC	3139.86	1304.74	2927.19	1453.77	2807.40	1392.91	517.4/267.8
sasc	31.30	191.00	27.28	191.20	26.96	191.20	3.0/1.4
simple_spi	41.00	287.00	36.96	308.22	35.89	293.36	8.0/3.1
spi	164.10	489.49	149.43	500.98	146.68	450.90	57.0/21.1
sqrt	1110.02	78002.24	1157.38	88015.07	1071.21	80961.45	221.8/206.7
systemcaes	511.67	784.00	483.75	709.03	447.57	682.83	112.8/58.1
systemcdes	138.84	530.29	131.06	488.85	125.89	487.52	75.8/25.0
tv80	351.11	907.86	340.61	846.12	330.87	833.94	159.6/63.8
usb_funct	695.57	702.87	646.23	668.08	622.62	664.39	207.9/105.6
usb_phy	23.63	163.82	22.54	163.29	21.39	158.68	9.3/1.9
Ave. Ratio	1.0000	1.0000	1.0189	1.0366	0.9088	0.9308	–

Each configuration/design runs cold with an empty private pool, then warm with the same pool and limits. Native caches restart; OS caches are not cleared. Both groups share the verified ABC batch. Times are t_c/t_w. Externally interrupted attempts are archived, excluded from timing totals, and retried with their prescribed cache state.

Completed and selected deadline outputs undergo CEC and independent serialized-PPA audits. A TO remains incomplete: the best legal checkpoint is selected, or the initial mapping. Snapshot costs count toward 1000 s; post-run validation does

not. Interrupted ABC runs retain the initial mapping. All 128 runs of the full configuration across the three parts finish within the post-mapping limit. Across all parts, cold/warm have 348/349 completions and 4/3 timeouts; 4/3 timeouts select improved checkpoints. Differing costs are stacked or separated by a slash, cold first. Define $\delta A_j = 100(A_j/A_{0,j} - 1)$ and $\delta D_j = 100(D_j/D_{0,j} - 1)$. Means include initial selections; ablations sum $\Delta A = \sum_j (A_{\mathrm{arm,j}} - A_{\mathrm{full,j}})$.

B. Area Recovery after AIG Preprocessing

Both preprocessing-experiment caps start from N_e (TABLE III). At one round, mean area changes are -5.04%; mean delay changes are -4.90% (cold/warm). At cap 6, these are -7.11% area and -7.43% delay, with 16/16 completed runs. Area decreases on 16 of sixteen; all selected outputs meet the initial bounds. The caps are measured independently.

For `arbiter`, area changes from 557.72 to 223.64 μm^2 and delay from 999.87 to 261.39 ps. The other fifteen average -3.59% area and -3.00% delay changes.

ASPDAC'25 costs are reconstructed from its reported starts and rounded changes [6]; its implementation is unavailable. Our final area is smaller on 14 designs, with mean area and delay differences of -9.12% and -6.92%. These approximate comparisons use different starts and cannot isolate post-mapping gains; our delay bound refers to our input.

C. Common-Input Comparison with ABC

Both tools receive the same direct mapping (TABLE II); ours uses one round. ABC's `mfs3 -a` does not enforce our initial-delay bound, so we distinguish raw and externally guarded outputs.

ABC completes 13 designs and times out on 3. On the ABC-completed subset, its mean area/delay changes are -3.58%/+11.02%, versus -7.06%/-6.61% for our verified selections. ePMapping completes 16/16 designs cold/warm. Its all-sixteen selected-output means are -6.96%/-5.47%, within the initial delay bounds.

Of the completed ABC outputs, 7 violate the delay bound and 6 satisfy both bounds. An external guard retains only feasible outputs, otherwise selecting the initial mapping; its paired means are -1.59% area and -0.15% delay. This guard leaves ABC unchanged.

Ratios in TABLE II use each tool's native initial costs: ABC import may normalize output buffers. Times exclude CEC; ABC includes import and writing, ours optimizer preparation through rebuild.

D. Operator Ablations and Efficiency

All nineteen arms use identical N_e starts and bounds (TABLE IV); L1 stays enabled. noSYN removes L3, noEGG removes L4, noDB removes lookup and database-derived mapper entries, and noCARE supplies full care. noCHAIN skips composition; noSHARE clears chain memo tables between roots. noNARROW removes narrow initialization; noPHASE disables physical phase reuse.

noSHANNON retains Davio only; noDAVIO retains Shannon only. noREUSE removes forward INV lookup but keeps backward cancellation; noPRICE removes contextual pricing while preserving physical reuse. noQUERY restores uncached inverse queries and dependency traversal. noTRUTH disables decoded simulation and dynamic-table reuse; the common scalar cut memo remains. allPIVOTS removes the pivot cap. cache64K uses 65536 native entries, noAIGREUSE disables pre-mapping AIG reuse, and noLIBSHARE copies libraries.

TABLE IV. Ablations on identical N_e starts; subscripts c, w denote cold/warm. ΔA (μm^2) sums area differences from full; W/B counts worse/better areas. Time/full divides total optimizer seconds by the same-mode full total. TO counts cold/warm deadlines; selected netlists are independently validated.

Arm	ΔA_c	ΔA_w	W/B$_c$	W/B$_w$	Time/full	TO
full	+0.00	+0.00	0/0	0/0	1.00/1.00	0/0
noSYN	+394.71	+394.71	16/0	16/0	0.73/0.59	0/0
noEGG	+3.89	+3.89	11/0	11/0	0.51/0.86	0/0
noDB	+1.62	+1.62	5/7	5/7	0.82/0.71	0/0
noCARE	+15.16	+15.16	13/1	13/1	0.95/0.91	0/0
noCHAIN	+334.08	+334.08	1/0	1/0	1.00/0.99	0/0
noSHARE	+334.08	+334.08	1/0	1/0	1.00/1.00	0/0
noNARROW	+0.92	+0.92	6/0	6/0	1.15/1.24	0/0
noPHASE	+243.82	+243.82	15/0	15/0	0.88/0.74	0/0
noSHANNON	+148.11	+148.11	15/0	15/0	0.97/0.95	0/0
noDAVIO	+39.52	+39.52	13/1	13/1	0.97/0.92	0/0
noREUSE	+231.82	+231.82	15/0	15/0	1.01/0.92	0/0
noPRICE	+163.59	+163.59	15/0	15/0	0.82/0.69	0/0
cache64K	+0.28	+0.00	1/0	0/0	1.18/1.37	1/0
noAIGREUSE	+0.00	+0.00	0/0	0/0	1.00/0.99	0/0
noLIBSHARE	+0.00	+0.00	0/0	0/0	1.03/1.03	0/0
noQUERY	+69.76	+68.63	2/0	2/0	1.67/2.15	2/2
noTRUTH	+0.00	+0.00	0/0	0/0	1.02/1.02	0/0
allPIVOTS	-36.93	-39.99	1/10	0/11	1.28/1.55	1/1

The noQUERY and noTRUTH time/full ratios are 1.67/2.15 and 1.02/1.02. allPIVOTS has ratio 1.28/1.55 and summed area differences -36.93/-39.99 μm^2. noTRUTH leaves every final area and delay unchanged. Unrestricted pivots recover more area overall but time out on `mem_ctrl` cold and warm (Fig. 2).

Removing Shannon and Davio gives summed area differences of +148.11 and +39.52 μm^2; removing forward reuse and live pricing gives +231.82 and +163.59 μm^2. All four removals increase area on `usb_phy`. Independent runs can follow different trajectories under care budgets and deadlines.

noDB improves 7 designs but changes both lookup and inner mapping. Ablation effects are not additive.

Removing L4 worsens area on 11 designs; its cold/warm time ratios are 0.51/0.86. L4 takes 49.4%/13.9% of recorded phase time in 16/16 completed full runs (CEC and other phases excluded). Pool hits avoid saturation and extraction; native caches restart per process. TO ratios compare elapsed budgets, not equal completed work. Shared-host scheduling affects elapsed time.

V. Conclusion

ePMapping combines shared AO21 blocks, cut decompositions, reusable arrival-independent profiles and live phase pricing under the initial delay bound. Query, truth-table and construction reuse avoid repeated work; support-ranked pivots limit candidate generation. The experiments measure area recovery and runtime separately. Finite candidate pools and pivot budgets can still omit useful implementations.

REFERENCES

[1] L. Hellerman, "A catalog of three-variable Or-Invert and And-Invert logical circuits," *IEEE Transactions on Electronic Computers*, vol. EC-12, no. 3, pp. 198–223, 1963.

[2] L. Amarú, P.-E. Gaillardon, and G. De Micheli, "Majority-Inverter graph: A novel data-structure and algorithms for efficient logic optimization," in *Proc. DAC*, 2014, pp. 1–6.

[3] ——, "Majority-Inverter graph: A new paradigm for logic optimization," *IEEE TCAD*, vol. 35, no. 5, pp. 806–819, 2016.

[4] A. T. Calvino, H. Riener, S. Rai, A. Kumar, and G. De Micheli, "A versatile mapping approach for technology mapping and graph optimization," in *Proc. ASPDAC*, 2022, pp. 410–416.

[5] A. Costamagna, A. Tempia Calvino, A. Mishchenko, and G. De Micheli, "Post-mapping resubstitution for area-oriented optimization," in *Proc. IWLS*, 2024.

[6] ——, "Area-oriented optimization after standard-cell mapping," in *Proc. ASPDAC*, 2025.

[7] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, "IMpress: Large integer multiplication expression rewriting for FPGA HLS," in *Proc. FCCM*, 2022, pp. 1–10.

[8] J. Cheng, S. Coward, L. Chelini, R. Barbalho, and T. Drane, "SEER: Super-optimization explorer for high-level synthesis using e-graph rewriting," in *Proc. ASPLOS*, 2024, pp. 1029–1044.

[9] S. Coward, T. Drane, and G. A. Constantinides, "ROVER: RTL optimization via verified e-graph rewriting," *IEEE TCAD*, vol. 43, no. 12, pp. 4687–4700, 2024.

[10] C. Chen, G. Hu, D. Zuo, C. Yu, Y. Ma, and H. Zhang, "E-Syn: E-graph rewriting with technology-aware cost functions for logic synthesis," in *Proc. DAC*, 2024.

[11] C. G. Nelson, "Techniques for program verification," Ph.D. dissertation, Stanford University, Stanford, CA, USA, 1980, aAI8011683.

[12] R. Tate, M. Stepp, Z. Tatlock, and S. Lerner, "Equality saturation: a new approach to optimization," in *Proc. POPL*, 2009, p. 264–276.

[13] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "egg: Fast and extensible equality saturation," *Proc. PACMPL*, vol. 5, no. POPL, pp. 1–29, 2021.

[14] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm finfet predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.

[15] M. Soeken, H. Riener, W. Haaswijk, E. Testa, B. Schmitt, G. Meuli, F. Mozafari, S.-Y. Lee, A. Tempia Calvino, and G. Marakkalage, Dewmini Sudara De Micheli, "The EPFL logic synthesis libraries," 2022.

[16] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. CAV*. Springer Berlin Heidelberg, 2010, pp. 24–40.